

Assignment Number: 8.5

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Lab Objectives:

- To introduce students to test-driven development (TDD)

Week4 -

Friday

using AI code generation tools.

- To enable the generation of test cases before writing code implementations.
- To reinforce the importance of testing, validation, and error handling.
- To encourage writing clean and reliable code based on AI-generated test expectations.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to write test cases for Python functions and classes.
- Implement functions based on test cases in a test-first development style.
- Analyze the completeness and coverage of AI-generated tests.
- Compare AI-generated and manually written test cases for quality and logic

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.
- Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

Example Assert Test Cases:

`assert is_valid_username("User123") == True`

`assert is_valid_username("12User") == False`

`assert is_valid_username("Us er") == False`

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

CODE :

```
C:\Users > k.Nikshitha > OneDrive > Desktop > Devops > is_valid.py > ...
1  def is_valid_username(username):
2      if len(username) < 5 or len(username) > 15:
3          return False
4      if not username[0].isalpha():
5          return False
6      for char in username:
7          if not (char.isalnum() or char == '_'):
8              return False
9      return True
10 # Assert test cases
11 assert is_valid_username("user_1") == True
12 assert is_valid_username("1user") == False
13 assert is_valid_username("user") == False
14 assert is_valid_username("this_is_a_very_long_username") == False
15 assert is_valid_username("valid_user") == True
16 assert is_valid_username("invalid-user") == False
17 assert is_valid_username("user123") == False
```

OUTPUT :

```
PS C:\Users\k.Nikshitha> & C:\Users\k.Nikshitha\AppData\Local\Programs\Python\Python314\python.exe c:\Users\k.Nikshitha\OneDrive\Desktop\Devops\is_valid.py
Traceback (most recent call last):
Traceback (most recent call last):
  File "c:\Users\k.Nikshitha\OneDrive\Desktop\Devops\is_valid.py", line 17, in <module>
    assert is_valid_username("user123") == False
AssertionError
```

Task Description #2 (Even–Odd & Type Classification – Apply

AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.
- Requirements:
 - o If input is an integer, classify as "Even" or "Odd".

- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

CODE :

```
C: > Users > k.Nikshitha > OneDrive > รูปภาพ > Screenshots > New folder > 2.py > ...
1  def classify_value(value):
2      if isinstance(value, int):
3          if value == 0:
4              return "Zero"
5          elif value % 2 == 0:
6              return "Even"
7          else:
8              return "Odd"
9      else :
10         return "Invalid Input"
11  # Example Assert test cases :
12  assert classify_value(0) == "Zero"
13  assert classify_value(2) == "Even"
14  assert classify_value(3) == "Odd"
15  assert classify_value(-4) == "Even"
16  assert classify_value(-5) == "Odd"
17  assert classify_value(3.14) == "Invalid Input"
```

OUTPUT :

```
PS C:\Users\k.Nikshitha> & C:\Users\k.Nikshitha\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/k.Nikshitha/OneDrive/รูปภาพ/Screenshots/New folder/2.py"
PS C:\Users\k.Nikshitha>
```

Task Description #3 (Palindrome Checker – Apply AI for

String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

- Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") ==

True

assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

CODE :

```
C: > Users > k.Nikshitha > OneDrive > รูปถ่าย > Screenshots > New folder > 3.py > ...
1  def is_palindrome(s):
2      cleaned = ''.join(c.lower() for c in s if c.isalnum())
3      return cleaned == cleaned[::-1]
4  #Assert test cases
5  assert is_palindrome("A man a plan a canal Panama") == True
6  assert is_palindrome("race a car") == False
7  assert is_palindrome("Was it a car or a cat I saw?") == True
```

OUTPUT :

```
PS C:\Users\k.Nikshitha> & C:\Users\k.Nikshitha\AppData\Local\Programs\Python\Python314\python.exe "C:\Users\k.Nikshitha\OneDrive\รูปถ่าย\Screenshots\New folder\3.py"
PS C:\Users\k.Nikshitha>
```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

Requirements:

- o Must contain @ and .
- o Must not start or end with special characters.
- o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

CODE :

```
1 def validate_email(email):
2     if '@' not in email or '.' not in email:
3         return False
4     if email[0] in ['@', '.'] or email[-1] in ['@', '.']:
5         return False
6     return True
7 #Example Assert Test Cases:
8 assert validate_email("user@example.com") == True
9 assert validate_email("userexample.com") == False
10 assert validate_email("user@.com") == False
```

OUTPUT :

```
PS C:\Users\k.Nikshitha> & c:/Users/k.Nikshitha/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/k.Nikshitha/OneDrive/gdmmw/Screenshots/new folder/3.py"
```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).

- Test Cases to Design:

o Normal case: 6 → True, 10 → False.

o Edge case: 1.

o Negative number case.

o Larger case: 28.

- Requirement: Validate correctness with assertions.

CODE :

```
C:\Users\k.Nikshitha\OneDrive\รูปถ่าย\Screenshots\New folder\5.py
1 def perfect_number_checker(num):
2     if num < 2:
3         return False
4     divisors_sum = 1 # Start with 1, which is a proper divisor
5     for i in range(2, int(num**0.5) + 1):
6         if num % i == 0:
7             divisors_sum += i
8             if i != num // i: # Add the complementary divisor if it's different
9                 divisors_sum += num // i
10    return divisors_sum == num
11 # Testcases using assert statements by using number 1
12 assert perfect_number_checker(6) == True # 6 is a perfect number
13 assert perfect_number_checker(28) == True # 28 is a perfect number
14 assert perfect_number_checker(12) == False # 12 is not a perfect number
15 assert perfect_number_checker(1) == False # 1 is not a perfect number
16
```

OUTPUT :

```
PS C:\Users\k.Nikshitha> & c:\Users\k.Nikshitha\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/k.Nikshitha/OneDrive/gumw/Screenshots/New folder/5.py"
PS C:\Users\k.Nikshitha>
```

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).

- Test Cases to Design:

- o Normal case: 12 → True, 15 → False.

- o Edge case: 1.

- o Negative number case.

- o Large case: 945.

Requirement: Validate correctness with unittest

CODE :

```
C: > Users > k.Nikshitha > OneDrive > รูปภาพ > Screenshots > New folder > 6.py > ...
1  def abundant_number_checker(num):
2      if num < 12: # The smallest abundant number is 12
3          return False
4      divisors_sum = 1 # Start with 1, which is a proper divisor
5      for i in range(2, int(num**0.5) + 1):
6          if num % i == 0: # If i is a divisor
7              divisors_sum += i
8              if i != num // i: # Add the complementary divisor if it's different
9                  divisors_sum += num // i
10         return divisors_sum > num
11
12 import unittest
13 class TestAbundantNumberChecker(unittest.TestCase):
14     def test_abundant_numbers(self):
15         self.assertTrue(abundant_number_checker(12)) # 12 is an abundant number
16         self.assertTrue(abundant_number_checker(18)) # 18 is an abundant number
17         self.assertTrue(abundant_number_checker(20)) # 20 is an abundant number
18
19     def test_non_abundant_numbers(self):
20         self.assertFalse(abundant_number_checker(6)) # 6 is not an abundant number
21         self.assertFalse(abundant_number_checker(28)) # 28 is not an abundant number
22         self.assertFalse(abundant_number_checker(1)) # 1 is not an abundant number
23
24 if __name__ == '__main__':
25     unittest.main()
```

OUTPUT :

```
PS C:\Users\k.Nikshitha>
> & c:\Users\k.Nikshitha\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/k.Nikshitha/OneDrive/gumw/Screenshots/New folder/6.py"
..
Ran 2 tests in 0.000s
OK
PS C:\Users\k.Nikshitha>
```

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).

- Test Cases to Design:

- o Normal case: 8 → True, 12 → False.

- o Edge case: 1.
- o Negative number case.
- o Large case: 546.

Requirement: Validate correctness with pytest.

CODE :

```
C: > Users > k.Nikshitha > OneDrive > Screenshots > New folder > 7.py > ...
1 def deficient_number_checker(num):
2     if num < 1:
3         return False # Deficient numbers are defined for positive integers
4     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
5     return divisors_sum < num
6 def test_positive_deficient_numbers():
7     # Test cases for deficient numbers
8     assert deficient_number_checker(1) == False # 1 is not a deficient number
9     assert deficient_number_checker(2) == True # 2 is a deficient number (divisors: 1)
10    assert deficient_number_checker(3) == True # 3 is a deficient number (divisors: 1)
11    assert deficient_number_checker(4) == True # 4 is a deficient number (divisors: 1, 2)
12    assert deficient_number_checker(5) == True # 5 is a deficient number (divisors: 1)
13    assert deficient_number_checker(6) == False # 6 is not a deficient number (it's perfect)
14    print("All tests passed!")
15
16 if __name__ == '__main__':
17     test_positive_deficient_numbers()
```

OUTPUT :

```
PS C:\Users\PC\Downloads\aiac> python -m pytest deficient_number_checker.pytest.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\PC\Downloads\aiac
collected 4 items

deficient_number_checker_pytest.py .... [100%]

===== 4 passed in 0.02s =====
```

Task 8 :

Write a function LeapYearChecker and validate its implementation

using 10 pytest test cases

CODE :

```

C: > Users > k.Nikshitha > Downloads > 8.py > test_year_2020
1  def LeapYearChecker(year):
3      return True
4  else:
5      return False
6  def test_leap_year_checker():
7      assert LeapYearChecker(2000) == True
8      assert LeapYearChecker(1900) == False
9  def test_year():
10     assert LeapYearChecker(2020) == True
11     assert LeapYearChecker(2021) == False
12  def test_century_year():
13     assert LeapYearChecker(1600) == True
14     assert LeapYearChecker(1700) == False
15  def test_non_leap_year():
16     assert LeapYearChecker(2019) == False
17     assert LeapYearChecker(2021) == False
18  def test_leap_year():
19     assert LeapYearChecker(2024) == True
20     assert LeapYearChecker(2028) == True
21  def test_edge_cases():
22     assert LeapYearChecker(0) == True
23     assert LeapYearChecker(-4) == True
24     assert LeapYearChecker(-100) == False
25  def test_year_2100():
26     assert LeapYearChecker(2100) == False
27     assert LeapYearChecker(2400) == True
28  def test_year_2000():
29     assert LeapYearChecker(2000) == True
30     assert LeapYearChecker(2001) == False
31  def test_year_1900():
32     assert LeapYearChecker(1900) == False
33     assert LeapYearChecker(1904) == True
34  def test_year_2020():
35     assert LeapYearChecker(2020) == True
36     assert LeapYearChecker(2021) == False

```

OUTPUT :

```

PS C:\Users\PC\Downloads\aiac> python -m pytest leapYearChecker_pytest.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\PC\Downloads\aiac
collected 10 items

leapYearChecker_pytest.py ..... [100%]

===== 10 passed in 0.05s =====

```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

CODE :


```

C: > Users > k.Nikshitha > Downloads > sum.py > test_sum_of_digits_with_mixed
1  def SumOfDigits(number):
2      total = 0
3      while number > 0:
4          total += number % 10
5          number //= 10
6      return total
7  def test_sum_of_digits():
8      assert SumOfDigits(123) == 6
9      assert SumOfDigits(456) == 15
10 def test_sum_of_digits_with_zero():
11     assert SumOfDigits(0) == 0
12     assert SumOfDigits(100) == 1
13 def test_sum_of_digits_with_negative():
14     assert SumOfDigits(-123) == 0
15     assert SumOfDigits(-456) == 0
16 def test_sum_of_digits_with_large_number():
17     assert SumOfDigits(987654321) == 45
18     assert SumOfDigits(1234567890) == 45
19 def test_sum_of_digits_with_single_digit():
20     assert SumOfDigits(5) == 5
21     assert SumOfDigits(9) == 9
22 def test_sum_of_digits_with_repeated_digits():
23     assert SumOfDigits(111) == 3
24     assert SumOfDigits(222) == 6
25 def test_sum_of_digits_with_mixed_digits():
26     assert SumOfDigits(12345) == 15
27     assert SumOfDigits(54321) == 15

```

OUTPUT :

```

PS C:\Users\PC\Downloads\aiac> python -m pytest sumOfDigits_pytest.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\PC\Downloads\aiac
collected 7 items

sumOfDigits_pytest.py ..... [100%]

===== 7 passed in 0.05s =====
PS C:\Users\PC\Downloads\aiac>

```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

CODE :

```

1 def SortNumbers(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if arr[j] > arr[j + 1]:
6                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
7     return arr
8
9 # test cases for SortNumbers function
10 def test_positive_numbers():
11     assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
12 def test_negative_numbers():
13     assert SortNumbers([-3, -1, -4, -2]) == [-4, -3, -2, -1]
14 def test_mixed_numbers():
15     assert SortNumbers([3, -1, 2, -5, 0]) == [-5, -1, 0, 2, 3]
16 def test_duplicates():
17     assert SortNumbers([4, 2, 2, 8, 3]) == [2, 2, 3, 4, 8]
18 def test_empty_array():
19     assert SortNumbers([]) == []
20 def test_single_element():
21     assert SortNumbers([42]) == [42]
22 def test_already_sorted():
23     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
24 def test_reverse_sorted():
25     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
26 def test_large_numbers():
27     assert SortNumbers([1000, 500, 2000, 1500]) == [500, 1000, 1500, 2000]
28 def test_floats():
29     assert SortNumbers([3.5, 2.1, 4.6, 1.0]) == [1.0, 2.1, 3.5, 4.6]
30 def test_mixed_types():
31     assert SortNumbers([1, 2.5, 1, 4.0]) == [1, 2.5, 3, 4.0]

```

```

31 def test_large_array():
32     assert SortNumbers(list(range(1000, 0, -1))) == list(range(1, 1001))
33 def test_array_with_zero():
34     assert SortNumbers([0, 5, 3, 1]) == [0, 1, 3, 5]
35 def test_array_with_negatives_and_zeros():
36     assert SortNumbers([-1, 0, -5, 3]) == [-5, -1, 0, 3]
37 def test_array_with_duplicates_and_negatives():
38     assert SortNumbers([-2, -2, 0, 1, 1]) == [-2, -2, 0, 1, 1]
39 def test_array_with_all_same_elements():
40     assert SortNumbers([7, 7, 7, 7]) == [7, 7, 7, 7]
41 def test_array_with_large_and_small_numbers():
42     assert SortNumbers([1000000, 1, 500000, 999999]) == [1, 500000, 999999, 1000000]
43 def test_array_with_floats_and_duplicates():
44     assert SortNumbers([-1.5, 2.3, -3.4, 0.0]) == [-3.4, -1.5, 0.0, 2.3]
45 def test_array_with_mixed_types_and_duplicates():
46     assert SortNumbers([3, 2.5, 1, 4.0, 2.5]) == [1, 2.5, 2.5, 3, 4.0]
47 def test_array_with_large_numbers_and_duplicates():
48     assert SortNumbers([1000, 500, 2000, 1500, 1000]) == [500, 1000, 1000, 1500, 2000]
49 def test_array_with_negatives_and_duplicates():
50     assert SortNumbers([-1, -1, -2, -3]) == [-3, -2, -1, -1]
51 def test_array_with_floats_and_duplicates():
52     assert SortNumbers([3.5, 2.1, 4.6, 1.0, 2.1]) == [1.0, 2.1, 2.1, 3.5, 4.6]
53 def test_array_with_mixed_types_and_large_numbers():
54     assert SortNumbers([3, 2.5, 1, 4.0, 1000000]) == [1, 2.5, 3, 4.0, 1000000]
55 def test_array_with_mixed_types_and_negatives():
56     assert SortNumbers([-1, 0, 1, -2, 2.5]) == [-2, -1, 0, 1, 2.5]
57 def test_array_with_mixed_types_and_floats():
58     assert SortNumbers([3, 2.5, 1, 4.0, -1.5]) == [-1.5, 1, 2.5, 3, 4.0]

```

OUTPUT :

```

PS C:\Users\PC\Downloads\aiac> python -m pytest sortNumbers_pytest.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\PC\Downloads\aiac
collected 25 items

sortNumbers_pytest.py ..... [100%]

===== 25 passed in 0.09s =====

```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

```

1 def ReverseString(s):
2     return s[::-1]
3 import unittest
4 class TestReverseString(unittest.TestCase):
5     def test_reverse_string(self):
6         self.assertEqual(ReverseString("hello"), "olleh")
7         self.assertEqual(ReverseString("world"), "dlrow")
8     def test_empty_string(self):
9         self.assertEqual(ReverseString(""), "")
10    def test_single_character(self):
11        self.assertEqual(ReverseString("a"), "a")
12    def test_palindrome(self):
13        self.assertEqual(ReverseString("madam"), "madam")
14    def test_numbers(self):
15        self.assertEqual(ReverseString("12345"), "54321")
16 if __name__ == '__main__':
17     unittest.main()

```

Code:

Output :

```

PS C:\Users\PC\Downloads\aiac> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/reverseString_unittest.py
.....
Ran 5 tests in 0.000s

OK

```

Task 12 :

Write a function `AnagramChecker` and validate its implementation using 10 unittest test cases.

Code:

```
1 def AnagramChecker(s):
2     return s[::-1]
3 import unittest
4 class TestAnagramChecker(unittest.TestCase):
5     def test_anagram_checker(self):
6         self.assertEqual(AnagramChecker("hello"), "olleh")
7         self.assertEqual(AnagramChecker("world"), "dlrow")
8     def test_empty_string(self):
9         self.assertEqual(AnagramChecker(""), "")
10    def test_single_character(self):
11        self.assertEqual(AnagramChecker("a"), "a")
12    def test_palindrome(self):
13        self.assertEqual(AnagramChecker("madam"), "madam")
14    def test_numbers(self):
15        self.assertEqual(AnagramChecker("12345"), "54321")
16    def test_special_characters(self):
17        self.assertEqual(AnagramChecker("!@#$%"), "%$#@!")
18    def test_mixed_characters(self):
19        self.assertEqual(AnagramChecker("Hello, World!"), "ldlrow ,olleh")
20    def test_unicode_characters(self):
21        self.assertEqual(AnagramChecker("你好"), "好你")
22    def test_long_string(self):
23        self.assertEqual(AnagramChecker("This is a long string for testing."), ".gnitset rof gnirts gnol a si siht")
24    def test_whitespace(self):
25        self.assertEqual(AnagramChecker(" "), " ")
26 if __name__ == '__main__':
27     unittest.main()
```

Output :

```
PS C:\Users\PC\Downloads\aiac> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/anagram_unittest.py
.....
-----
Ran 10 tests in 0.001s

OK
```

Task 13 :

Write a function `ArmstrongChecker` and validate its implementation using 8 unittest test cases.

Code:

```
1 def ArmstrongChecker(num):
2     num_str = str(num)
3     num_digits = len(num_str)
4     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
5     return armstrong_sum == num
6 import unittest
7 class TestArmstrongChecker(unittest.TestCase):
8     def test_armstrong_number(self):
9         self.assertTrue(ArmstrongChecker(153))
10    def test_non_armstrong_number(self):
11        self.assertFalse(ArmstrongChecker(123))
12    def test_single_digit_armstrong(self):
13        self.assertTrue(ArmstrongChecker(7))
14    def test_two_digit_non_armstrong(self):
15        self.assertFalse(ArmstrongChecker(10))
16    def test_four_digit_armstrong(self):
17        self.assertTrue(ArmstrongChecker(9474))
18    def test_five_digit_non_armstrong(self):
19        self.assertFalse(ArmstrongChecker(12345))
20    def test_six_digit_armstrong(self):
21        self.assertTrue(ArmstrongChecker(548834))
22    def test_large_non_armstrong(self):
23        self.assertFalse(ArmstrongChecker(1000000))
24 if __name__ == '__main__':
25     unittest.main()
```

```
PS C:\Users\PC\Downloads\aiac> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/armstrongNumber_unittest.py
.....
-----
Ran 8 tests in 0.001s

OK
```